

Manual breve de despliegue y ejecución local

Fecha: 28/08/2026

Versión 1.0

1. Descripción general

El proyecto **Canchas Deportivas** está desarrollado bajo una arquitectura de microservicios y microfrontends.

La solución está compuesta por:

- PostgreSQL: base de datos principal.
- **ms-usuarios**: gestión de usuarios, roles y autenticación mediante JWT.
- **ms-canchas**: gestión de deportes, canchas y horarios de atención.
- **ms-reservas**: gestión de reservas, disponibilidad y cancelaciones.
- **ms-reportes**: generación de reportes básicos de ocupación y reservas por período.
- API Gateway: punto de entrada único para las peticiones del frontend.
- Frontend Host: aplicación principal.
- Microfrontends: módulos independientes integrados mediante Module Federation.

Los microservicios utilizan PostgreSQL y se comunican mediante APIs HTTP.

2. Requisitos previos

Para ejecutar el proyecto se requiere tener instalado:

- Docker Desktop.
- Docker Compose.
- Python 3.13 o una versión compatible con el proyecto.
- Node.js y npm para ejecutar los proyectos frontend.
- Git, en caso de clonar el proyecto desde el repositorio.

Verificar las instalaciones:

```
docker --version
docker compose version
python --version
node --version
npm --version
```

3. Estructura general del proyecto

La estructura principal es:

```
backend/
├── database/
├── ms-usuarios/
├── ms-canchas/
├── ms-reservas/
├── ms-reportes/
├── api-gateway/
└── docker-compose.yml

frontend/
  Apps/
    ├── shell/
    ├── mf-administracion/
    ├── mf-reservas/
    └── mf-reportes/
```

Api Gateway/

4. Configuración de variables de entorno

Cada microservicio utiliza un archivo `.env` para configurar las conexiones y parámetros necesarios.

En ejecución local, las URLs utilizan `localhost` o `127.0.0.1`.

Ejemplo para `ms-reportes`:

```
CANCHAS_SERVICE_URL=http://127.0.0.1:8002
RESERVAS_SERVICE_URL=http://127.0.0.1:8003
```

```
SECRET_KEY=microserviciousuarios
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

Cuando los servicios se ejecutan dentro de Docker, la comunicación entre contenedores utiliza los nombres de los servicios definidos en `docker-compose.yml`.

Ejemplo:

```
CANCHAS_SERVICE_URL=http://ms-canchas:8002
RESERVAS_SERVICE_URL=http://ms-reservas:8003
```

El archivo `.env` no debe incluirse en el repositorio si contiene credenciales o secretos.

5. Despliegue completo con Docker

5.1. Ubicarse en la carpeta principal

Abrir una terminal en la carpeta `backend`:

```
cd backend
```

5.2. Construir las imágenes

Ejecutar:

```
docker compose build
```

Este comando construye las imágenes correspondientes a los servicios definidos en Docker Compose.

5.3. Levantar todos los contenedores

Ejecutar:

```
docker compose up -d
```

La opción `-d` permite ejecutar los contenedores en segundo plano.

5.4. Verificar los contenedores

Ejecutar:

```
docker compose ps
```

Todos los servicios necesarios deben aparecer con estado `Up` o `running`.

6. Base de datos PostgreSQL

PostgreSQL se ejecuta mediante Docker.

Los scripts SQL ubicados en la carpeta de inicialización crean los esquemas, tablas y datos iniciales necesarios para el sistema.

La base de datos contiene los esquemas correspondientes a los microservicios, por ejemplo:

```
usuarios  
canchas  
reservas
```

Para consultar los logs de PostgreSQL:

```
docker compose logs postgres
```

Para ingresar al contenedor:

```
docker exec -it backend-postgres psql -U postgres
```

El nombre del contenedor puede variar según la configuración utilizada en `docker-compose.yml`.

7. Microservicios

Los microservicios se ejecutan como contenedores independientes.

La distribución utilizada durante el desarrollo es:

Servicio	Puerto
API Gateway	8000*
ms-usuarios	8001
ms-canchas	8002
ms-reservas	8003
ms-reportes	8004*

* Los puertos del API Gateway y frontend deben ajustarse a los definidos finalmente en [docker-compose.yml](#).

Los endpoints de cada microservicio pueden comprobarse mediante Swagger.

Ejemplo:

<http://localhost:8001/docs>

<http://localhost:8002/docs>

<http://localhost:8003/docs>

Para [ms-reportes](#):

<http://localhost:<puerto-reportes>/docs>

8. Autenticación

El sistema utiliza autenticación mediante JWT.

Primero se debe iniciar sesión mediante el endpoint correspondiente de [ms-usuarios](#) o mediante el API Gateway.

La respuesta proporciona un token de acceso.

En Swagger se utiliza el botón:

Authorize

y se introduce:

Bearer <TOKEN>

Los roles principales son:

- **usuario**: puede consultar y administrar sus propias reservas según las reglas del sistema.
 - **administrador**: dispone de permisos administrativos y puede consultar o cancelar reservas de otros usuarios.
-

9. Ejecución local sin Docker

Para ejecutar los microservicios directamente desde Python se utiliza un entorno virtual independiente para cada servicio.

Ejemplo para **ms-usuarios**:

```
cd ms-usuarios
```

Crear el entorno virtual:

```
python -m venv .venv
```

Activarlo:

```
.\.venv\Scripts\Activate.ps1
```

Instalar las dependencias:

```
pip install -r requirements.txt
```

Ejecutar el servicio:

```
uvicorn app.main:app --reload --port 8001
```

Para los demás microservicios se repite el procedimiento utilizando el puerto correspondiente.

Ejemplo:

```
uvicorn app.main:app --reload --port 8002
```

```
uvicorn app.main:app --reload --port 8003
```

```
uvicorn app.main:app --reload --port 8004
```

10. Ejecución de pruebas

Cada microservicio contiene pruebas unitarias.

Para ejecutarlas:

```
pytest -v
```

Las pruebas utilizan mocks cuando corresponde, por lo que no es necesario crear una base de datos adicional exclusivamente para ejecutar los tests.

Ejemplo:

```
ms-usuarios
```

```
pytest -v
```

```
ms-canchas
```

```
pytest -v
```

```
ms-reservas
```

```
pytest -v
```

```
ms-reportes
```

```
pytest -v
```

El resultado esperado es que todas las pruebas finalicen con estado:

```
PASSED
```

11. Ejecución del frontend

Para cada aplicación frontend se deben instalar primero las dependencias:

```
npm install
```

Posteriormente se inicia el proyecto:

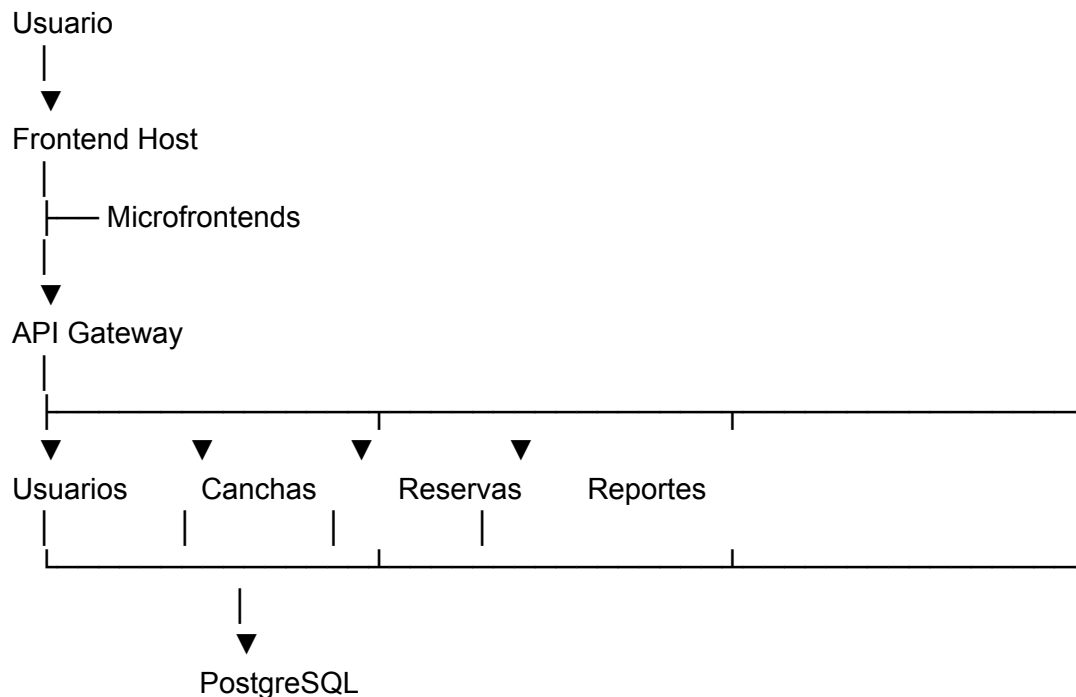
```
npm run dev
```

Los microfrontends deben estar ejecutándose para que el Frontend Host/Shell pueda cargarlos mediante Module Federation.

El Frontend Host actúa como aplicación principal y consume los módulos remotos configurados en Module Federation.

12. Flujo de ejecución completo

En un despliegue completo, el flujo general es:



El frontend se comunica con el API Gateway, mientras que el Gateway dirige las solicitudes al microservicio correspondiente.

ms-reportes consume información de **ms-canchas** y **ms-reservas** mediante HTTP para generar los reportes.

13. Comprobación rápida del sistema

Una vez levantados todos los contenedores:

1. Comprobar Docker

`docker compose ps`

2. Comprobar los microservicios

Abrir Swagger:

`http://localhost:8001/docs`

`http://localhost:8002/docs`

`http://localhost:8003/docs`

y el puerto correspondiente de `ms-reportes`.

3. Probar autenticación

Iniciar sesión con un usuario de prueba y obtener el JWT.

4. Probar roles

Verificar las operaciones utilizando:

- Usuario normal.
- Administrador.

5. Probar reservas

Comprobar:

- Creación de reservas.
- Validación de horarios.
- Prevención de reservas solapadas.
- Consulta de reservas.
- Consulta de disponibilidad.
- Cancelación de reservas.
- Restricciones según usuario y administrador.

6. Probar reportes

Comprobar:

- Ocupación por cancha.
 - Cantidad de reservas por período.
-

14. Detener el proyecto

Para detener los contenedores sin eliminarlos:

```
docker compose stop
```

Para detenerlos y eliminarlos:

```
docker compose down
```

Para volver a iniciar:

```
docker compose up -d
```

Si se modificó el código y es necesario reconstruir las imágenes:

```
docker compose up -d --build
```

15. Reinicialización de la base de datos

En caso de necesitar una inicialización completamente limpia de PostgreSQL, se pueden detener los servicios y eliminar el volumen asociado a la base de datos.

```
docker compose down -v
```

Posteriormente:

```
docker compose up -d --build
```

Los scripts SQL de inicialización volverán a ejecutarse sobre la nueva instancia de PostgreSQL.

Nota: este procedimiento elimina los datos almacenados en el volumen de PostgreSQL y debe utilizarse únicamente cuando se requiera una base de datos limpia.

16. Resumen de comandos principales

Levantar todo

`docker compose up -d --build`

Ver estado

`docker compose ps`

Ver logs

`docker compose logs -f`

Detener

`docker compose down`

Reiniciar

`docker compose up -d`

Ejecutar pruebas

`pytest -v`

Crear entorno virtual

`python -m venv .venv`

Activar entorno virtual en Windows

`.\.venv\Scripts\Activate.ps1`

Instalar dependencias

`pip install -r requirements.txt`

Ejecutar FastAPI localmente

`uvicorn app.main:app --reload --port <PUERTO>`

17. Credenciales de prueba

Para las pruebas del sistema se dispone de usuarios de prueba definidos mediante los scripts de seed de la base de datos.

Se recomienda utilizar:

- Un usuario con rol `usuario`.

```
{  
  "email": "usuario@test.com",  
  "password": "123456"  
}
```

- Un usuario con rol `administrador`.

```
{  
  "email": "admin@test.com",  
  "password": "admin123"  
}
```

Las credenciales reales deben mantenerse únicamente en la configuración de desarrollo y no deben incluirse en repositorios públicos.